Deploying SC Nav for your org

A step-by-step guide for an org that wants to run its own SC Nav instance on a cheap VPS. Written for **volunteers, not sysadmins**: everything after the initial install happens in a web browser, and there is no ongoing command-line work except one nightly backup job you set up once.

Status: ✅ current · written 2026-08-04 · DNS migration expanded 2026-08-06

Time to first login: about 2 hours of actual work, most of it waiting on Discord and Cloudflare forms. You do not need to know Docker, Linux, or how to configure a web server. The one thing that can stretch past that is the DNS move in step 5 — the registrar controls that clock, not you, and it's usually minutes but can be a day.

---

1. Who does what

Split the work so one person isn't a permanent bottleneck.

Role	How many	What they do	Needs server access?
Server owner	1 (a backup is wise)	Does the one-time install below; clicks "redeploy" when there's an update	Yes
Org admins	2–4	Everything day-to-day: org settings, POI quality, member roles, clearing bad data	**No** — all in the app's ADMIN panel
Members	everyone	Use the app; run the watcher on their gaming PC	No

The important part: **once installed, ~95% of administration happens inside the app itself**, not on the server. Org admins never need to log into the VPS.

---

2. What to buy

Network Solutions NVMe 4 — 2 vCPU / 4 GB RAM / 100 GB NVMe (~$4.18/mo intro).

Buy it through the Docker/DevOps page, not the plain VPS page. They're the same product and the same price, but the Docker page's checkout lets you preselect **Docker + Portainer**, which saves you an install and gives you a web dashboard to manage everything. Choose **Ubuntu** as the OS.

Why this tier, for an org of ~180 members with ~90 online at peak:

- **RAM:** the app uses ~160 MB at rest. Its one memory cost that scales with people is the breadcrumb trail (~2 MB per active member, worst case). Even with your *entire* org online at once with maxed trails, you'd use about 930 MB — under a quarter of 4 GB.

- **CPU:** 2 cores is the sweet spot. The app is single-process by design and GIL-bound, so it can only use about one core no matter what you buy — the second core serves Docker, the tunnel, and the network stack. **NVMe 8 would be exactly as fast**; only buy it if you plan to run other services (a Discord bot, a wiki) on the same box.
- **Disk:** 100 GB is ~100× what this needs.

Do not buy: the cPanel add-on (unnecessary — Portainer is free and already there), a dedicated IP (the Cloudflare Tunnel below needs no inbound connection), or any DDoS/firewall add-on (Cloudflare covers it).

> ⚠️ **Budget warning.** That $4.18 is an introductory rate on a multi-year prepaid term. Renewal is meaningfully higher. Check the renewal price in the cart *before* committing, and put the year-2 number in the org's budget.

---

3. Before you start: gather these

Collect all of this first — the install goes smoothly if you're not hunting for IDs halfway through.

- [ ] **A domain you control**, so the app can live at e.g. `nav.yourorg.com`. The domain's **DNS** has to be moved to Cloudflare (free) in step 5 — you keep the domain at its current registrar, only the DNS answering moves.
- [ ] **The registrar login** for that domain (Network Solutions, GoDaddy, Namecheap — whoever the org bought it from). Not the web-hosting login; the *registrar* account. Chasing this down later is the #1 stall.
- [ ] **An answer to: does org email run on this domain?** If anyone receives mail at `@yourorg.com`, read step 5 in full before touching anything.
- [ ] **A Cloudflare account** (free tier is fine, email-verified).
- [ ] **Discord Developer Mode on**: Discord → Settings → Advanced → Developer Mode.
- [ ] **Your Discord server ID**: right-click the server → Copy Server ID.
- [ ] **Admin user IDs**: right-click each admin's name → Copy User ID. Collect 2–4.
- [ ] *(Optional)* **A member role ID** if you want to restrict login to one role — right-click the role → Copy Role ID. Leave blank to allow any member of the server.
- [ ] **A session secret**: any random 64-character hex string. Generate one at a password generator, or run `openssl rand -hex 32` if you have a terminal.

---

4. Create the Discord sign-in app

The app has **no Discord bot** — it only verifies that whoever logs in is a member of your server.

1. Go to the Discord Developer Portal → **New Application**. Name it after your org.
2. Left sidebar → **OAuth2**.
3. Under **Redirects**, click **Add Redirect** and enter exactly: `https://nav.yourorg.com/auth/callback` (substitute your real subdomain). **Save Changes.**
4. Copy the **Client ID** — you'll need it shortly.
5. Click **Reset Secret**, copy the **Client Secret**, and paste it somewhere safe. Discord shows it only once. **Treat it like a password.**

> The redirect URL must match your real public URL **character for character**, including `https://` and the `/auth/callback` path. A mismatch here is the single most common cause of "login just bounces back" — see the runbook.

---

5. Move the domain's DNS to Cloudflare

This is the only step that touches something the org already depends on, so it gets the most words. Everything else in this guide is additive; this one changes an existing system. Read the whole section before you click anything.

5.0 What this actually is (and isn't)

Is: changing which servers answer the question "what is `yourorg.com`?" from Network Solutions' nameservers to Cloudflare's.

Is not:

- **Not a domain transfer.** Network Solutions stays your **registrar**. You still own the domain there, still renew it there, still pay them. Nothing about ownership, expiry, or WHOIS changes. (Cloudflare will offer to sell you a transfer. You don't need one. Decline.)
- **Not a website move.** The org's existing site keeps running on the same host. You're copying its DNS records to a new place that hands out the same answers.

- **Not optional, unfortunately.** The Cloudflare Tunnel in step 6 creates its own `nav.yourorg.com` record automatically, and it can only do that if Cloudflare is authoritative for the domain. Cloudflare's "partial" setup — where you keep DNS elsewhere and point one CNAME at them — is a **Business plan feature**, so on the free tier the whole domain comes over.

> **Escape hatch.** If the org's website and email live on this domain and nobody feels good about moving it: **register a separate domain for the app** (~$10–15/yr) and run the whole of step 5 against that one instead. A fresh domain has no records to break. `scnav-yourorg.com` is not as pretty as `nav.yourorg.com`, and it is a completely legitimate choice — moving a live org's email is the one thing in this guide that can actually hurt.

5.1 Inventory the existing DNS records — do not skip this

Cloudflare will try to import your records automatically. **Its scan is best-effort and routinely misses things.** It isn't authoritative for the zone yet, so it can't just list the records — it queries a list of common names and keeps what answers. Anything unusually named is invisible to it.

So write down what's actually there, first:

1. Log in at https://www.networksolutions.com/my-account/login.
2. Left sidebar → **Domains** → click your domain (a single-domain account goes straight to the overview).
3. **Advanced Tools** → **DNS** (or **Advanced DNS**) → **Manage**.
4. Copy **every** record into a text file: type, name/host, value, TTL, and priority for MX. Screenshot each page too — screenshots capture what you forget to transcribe.

Records that matter and are easy to miss:

Look for	Why it matters
`MX` (any)	**Org email.** Miss one and mail stops arriving.
`TXT` at root starting `v=spf1`	SPF — miss it and your mail lands in spam
`TXT` at `_dmarc`	DMARC — same
`TXT` / `CNAME` at `*._domainkey`, `selector1._domainkey`, `google._domainkey`	DKIM signing keys
`CNAME` `autodiscover`, `autoconfig`, `enterpriseregistration`	Outlook / M365 client setup
`A` / `CNAME` `mail`, `webmail`, `smtp`, `imap`	Where mail clients connect
`A` / `CNAME` at root (`@`) and `www`	The org's website itself
`TXT` verification strings (Google, Microsoft, Facebook)	Removing one un-verifies the org somewhere
`SRV` records	Discord vanity invites, VoIP, Minecraft servers

Save that file in the org's shared drive. **It is your rollback plan**, and the person who needs it at 11pm may not be you.

> If the record list mentions Network Solutions' own web or email hosting products, the org is a *customer* of those and the records must be copied exactly. Moving DNS does not cancel that hosting; deleting its records effectively does.

5.2 Lower the TTLs a day ahead (optional, 5 minutes, worth it)

On that same NS DNS screen, set the **TTL** on your existing records to **300** (5 minutes) and save, ideally ~24 hours before the switch. TTL is how long the rest of the internet is allowed to cache an old answer. Lowering it first means that if something goes wrong, your rollback takes effect in five minutes instead of a day.

Skip this only if you're doing everything in one sitting and can accept a slower undo.

5.3 Turn off DNSSEC at Network Solutions — before anything else

> 🚨 **This is the one that takes a domain completely offline.** DNSSEC cryptographically signs your DNS. The signature lives with the registrar and points at the *old* nameservers. Change nameservers with it still on and resolvers conclude your domain is being forged — not "slow", not "some people see the old site", but `SERVFAIL` **for everyone, website and email both**, until it's fixed.

Check whether it's on. From any Mac or Linux terminal:

```
dig DS yourorg.com +short
```

Empty output = DNSSEC is off, nothing to do, move to 5.4. Any output (a line of numbers and a hash) = it's on, and you must turn it off:

1. NS account → your domain → **Advanced Tools** → **DNSSEC** → disable / remove the DS records.
2. **Wait about an hour**, then re-run the `dig` above and confirm it's empty.
3. Only then continue.

You can turn DNSSEC back on later, from Cloudflare, in 5.8.

5.4 Add the domain to Cloudflare

1. https://dash.cloudflare.com → **Add a domain** (newer accounts label it **Onboard a domain**).
2. Enter the **apex** domain — `yourorg.com`. **Not** `nav.yourorg.com`. Cloudflare works one whole domain at a time; the subdomain comes later, by itself, in step 6.
3. Choose the **Free** plan.
4. Let it scan, then **check its work against your 5.1 file, line by line.** Add anything it missed by hand (**Add record**). This is the single highest value five minutes in this guide.
5. **Set proxy status deliberately:**
   - **MX records must be ⚪ DNS-only.** Cloudflare grays them out for you.
   - **So must any hostname mail uses** — `mail`, `smtp`, `imap`, `webmail`. Cloudflare does **not** always do this for you, and a proxied `mail` record hands out Cloudflare's IP to mail servers, which silently breaks delivery.
   - **The org's existing website records:** if you're unsure, set them ⚪ **DNS-only** as well. That makes this move a pure like-for-like swap — same answers, new nameservers. You can switch them to 🟠 proxied later, on purpose, when you have time to test.

6. **Do not create a `nav` record.** Step 6 creates it automatically, and a hand-made one will conflict with it.

5.5 Copy your two Cloudflare nameservers

They're on the domain's **Overview** page, and they look like:

```
bella.ns.cloudflare.com
carl.ns.cloudflare.com
```

The name pair is **assigned to your specific domain** and cannot be changed — use exactly the two Cloudflare shows you. Nameservers copied from a tutorial or from another org's setup will not activate your domain.

5.6 Point Network Solutions at Cloudflare

1. NS account → left sidebar **Domains** → your domain → **Settings**.
2. Scroll to **Advanced Tools** → find **Nameservers (DNS)** → click **Manage**.
3. Click **Continue** on the confirmation pop-up.
4. **Replace all existing nameservers** with the two from 5.5. Delete NS's own entries (typically `ns1.worldnic.com` / `ns2.worldnic.com`) completely.
5. **Save.**

> ⚠️ **Do not click "Custom Nameservers"** in that same Advanced Tools section. Despite the name, that's the *vanity nameserver* feature — it registers glue records like `ns1.yourorg.com` and is the wrong tool entirely. You want the plain **Manage** link next to Nameservers (DNS).

> ⚠️ **No mixing.** Exactly two entries, both Cloudflare's. Leaving one old nameserver in the list is the most common reason a zone sits on "Pending" forever — Cloudflare refuses to activate a domain it doesn't fully control.

> ⚠️ **You lose the NS DNS editor.** Once nameservers point elsewhere, Network Solutions' Advanced DNS screen stops having any effect (their help text says as much). From here on, **all DNS edits happen in Cloudflare.** Tell whoever normally manages the org's site.

5.7 Wait for "Active"

Cloudflare checks after 60 seconds, then at widening intervals, and emails you when it succeeds. The domain's Overview goes:

Pending Nameserver Update → Active

In practice this is usually **minutes to a couple of hours**. Network Solutions documents up to **24–72 hours** as the worst case; quote that number to anyone who asks, then expect much better.

Check it yourself instead of refreshing the dashboard:

```
dig NS yourorg.com +short          # should list your two Cloudflare nameservers
```

or use https://www.whatsmydns.net (pick record type **NS**) to see it landing worldwide.

You do not have to wait here. Go do steps 6, 7 and 8 now — the tunnel, the Portainer deploy, all of it. The site just won't load until this flips Active.

5.8 After it goes Active — verify, then finish up

Do these the same day, while you still remember what you changed:

- [ ] **Load the org's existing website.** It should look identical.
- [ ] **Send a test email to an `@yourorg.com` address from outside** (a personal Gmail), and send one *from* the org address. Both directions.
- [ ] **Re-enable DNSSEC**, if you turned it off in 5.3 — now from Cloudflare: **DNS → Settings → DNSSEC → Enable DNSSEC**. Cloudflare gives you a DS record; paste it into NS's DNSSEC screen. *Optional* — if this feels like one step too many, leaving DNSSEC off is a perfectly normal way to run a domain.
- [ ] **If you set any website records to 🟠 proxied**, go to **SSL/TLS → Overview** and choose **Full (strict)**. The default can be "Flexible", which serves your site over an unencrypted hop and breaks logins on some backends. (If you left everything DNS-only per 5.4, this setting doesn't apply to you — the tunnel handles its own encryption.)

5.9 If it goes wrong: rollback

Go back to the same NS **Manage** screen from 5.6 and put the original nameservers back (`ns1.worldnic.com` / `ns2.worldnic.com`, or whatever your 5.1 notes recorded). DNS reverts to exactly what Network Solutions was serving before — which is why 5.1 and 5.2 exist. Nothing is lost; the Cloudflare zone just sits inactive until you try again.

6. Put it on the internet (Cloudflare Tunnel)

This is the step that removes the most long-term maintenance. A tunnel makes an **outbound-only** connection from your server to Cloudflare, which means:

- No open ports, no firewall rules to manage
- No TLS certificate to install or renew — ever
- Your server's IP is never exposed

Steps:

1. Go to **Cloudflare Zero Trust** → **Networks** → **Tunnels** → **Create a tunnel** → choose **Cloudflared**. Name it `sc-nav`.
2. On the install screen, **ignore the install commands** — you don't need them. Just copy the long **token** string. That's your `CLOUDFLARE_TUNNEL_TOKEN`. Treat it like a password: it is the credential that lets a machine publish traffic on your domain.
3. Under **Public Hostnames**, add:
   - **Subdomain:** `nav` · **Domain:** `yourorg.com` (picked from a dropdown of your Cloudflare domains — if yours isn't listed, step 5 hasn't gone Active yet)
   - **Service type:** `HTTP` · **URL:** `sc-nav:8765`

   That `sc-nav:8765` is the app's name inside Docker — not a typo, and not an IP address. It stays `HTTP`, not `HTTPS`: the leg from Cloudflare to your server is already encrypted by the tunnel itself.
4. Save. Saving here **creates the `nav.yourorg.com` DNS record for you** — you won't find it in the DNS tab as a normal record, it shows as a tunnel route. That's expected.
5. The tunnel will show **inactive** until you finish step 7. That's expected too — nothing is running on the server end yet.

---

7. Deploy the app

Log into Portainer at the address Network Solutions gave you (usually `https://your-server-ip:9443`) and set your admin password on first visit.

1. Left sidebar → **Stacks** → **Add stack**.

2. **Name:** `sc-nav`
3. **Build method:** choose **Repository**.
   - **Repository URL:** `https://github.com/ByteCollectiveIO/sc-nav`
   - **Reference:** `refs/heads/stable` ← **not** `main`, see the note below
   - **Compose path:** `docker-compose.yml`

> **Why `stable`?** Because `main` is the development trunk — it moves several times a day and any given commit on it is mid-thought. `stable` only ever moves to a published release. Pointing here means your **Pull and redeploy** click always lands on a version that was tagged, released, and announced, whenever you happen to click it. (Pinning a specific tag like `refs/tags/v1.3.0` also works and is how you'd hold a version deliberately — but a tag never moves, so you'd have to hand-edit this field for every update.)

4. Scroll to **Environment variables** → **Add an environment variable** for each row below. This is why we gathered everything in step 3.

Name	Value
`DISCORD_CLIENT_ID`	from step 4
`DISCORD_CLIENT_SECRET`	from step 4 — keep private
`OAUTH_REDIRECT_URI`	`https://nav.yourorg.com/auth/callback`
`ORG_GUILD_ID`	your Discord server ID
`ADMIN_IDS`	admin user IDs, comma-separated, no spaces
`SESSION_SECRET`	your random hex string
`SC_NAV_PUBLIC_URL`	`https://nav.yourorg.com`
`COOKIE_SECURE`	`true`
`ORG_MEMBER_ROLE_ID`	a role ID, or leave blank
`CLOUDFLARE_TUNNEL_TOKEN`	the token from step 6
`STRATA_API_KEY`	leave blank (optional feature, add later)

> `COOKIE_SECURE` accepts `true` / `1` / `yes` / `on` interchangeably (and `false` / `0` / `no` / `off` for plain-HTTP local dev only). Anything it doesn't recognize — including a blank value — is read as **secure**, on purpose: this setting must never fail open. Just write `true`.
>
> One optional variable isn't in the table: `SC_NAV_UPDATE_REPO`, which controls which repo the admin **Server version** panel checks for releases. Leave it unset to watch upstream; set it to your own fork's `owner/name` to watch yours, or set it **empty** to switch the update check off entirely (no outbound call to GitHub at all).

5. Click **Deploy the stack**. The first deploy builds the app from source and takes **3–6 minutes** on this tier. Later ones are faster.
6. Back in Cloudflare, the tunnel should flip to **Healthy** within a minute.
7. Visit `https://nav.yourorg.com`. You should see the login splash.

---

8. First login and day-one setup

Log in with Discord. Because your ID is in `ADMIN_IDS`, you'll have the ADMIN panel.

Do this first — the app looks broken without it:

> 🚨 **Turn on the POI catalogs.** Go to **ORG SETTINGS** and enable both **starmap POIs** and **wiki POIs**. They ship **off** by default, and with them off the app knows about only **29 locations instead of 2,154**. New deployments regularly mistake this for a broken install. The wiki catalog is also *required* for the trade planner's ILLICIT cargo filter to work at all.

Then, while you're in ORG SETTINGS:

- **Branding** — set your org name (appears on the login splash and launcher).
- **Message of the day** — optional banner for members.
- **UEX price data** — leave the refresh interval at the default 6 hours. There is a hard 2-hour minimum; don't try to go below it.
- **Admins** — promote your other org admins here so you're not the only one.

- **Discord notifications** — optional, but it's what makes the app feel alive in a server that already exists. There are seven independent categories (`events` , `marketplace` , `goals` , `records` , `lfg` , `pirates` , `survey`); each takes a **Discord webhook URL** and is on exactly when it has one. Create the webhooks in Discord (channel → Edit Channel → Integrations → Webhooks) and paste each into its row. Point them at *different* channels — event reminders and marketplace posts in one feed gets muted fast. No bot, no extra permissions; a webhook can only post to the channel that made it.

Finally, **generate a watcher token** (Settings page) and walk one member through the Setup page: they download a pre-configured watcher, run `run_watcher.bat` , and type `/showlocation` in game. When their position shows up on your map, the install is confirmed end to end.

---

9. Backups — the one thing that needs a terminal

Everything members create — custom POIs, survey marks, trade history, inventory, goals — lives in a single database file. **It is irreplaceable.** The repo ships a backup script; you set it up once and then forget it.

SSH into the server (Network Solutions provides the details) and run:

```
sudo apt update && sudo apt install -y sqlite3 git
sudo git clone https://github.com/ByteCollectiveIO/sc-nav /opt/sc-nav
docker volume ls | grep sc-nav        # note the exact volume name
```

Then `sudo crontab -e` and add one line (substituting the volume name you just saw):

```
0 4 * * * SC_NAV_DATA=/var/lib/docker/volumes/sc-nav_sc-nav-data/_data
SC_NAV_BACKUP_DIR=/var/backups/sc-nav /opt/sc-nav/server/deploy/backup_db.sh >>
/var/log/sc-nav-backup.log 2>&1
```

That takes a safe copy every night at 4 AM without stopping the app, and keeps the newest 14.

Backups on the same server don't protect you from losing the server. Once a month, pull a copy down to your own machine. From your laptop (not the server):

```
scp root@your-server-ip:/var/backups/sc-nav/$(ssh root@your-server-ip 'ls -1t
/var/backups/sc-nav | head -1') ~/Downloads/
```

or just `ssh` in, `ls -1t /var/backups/sc-nav | head -1` to see the newest name, and `scp` that one file. Drop it in the org's shared drive. Put a recurring reminder on the org calendar — this is the step orgs skip and regret.

Test a restore once, before you need one. Follow the restore recipe in the runbook against a copy on your own machine (`sqlite3 restored.db ".tables"` is enough to prove the file is intact). An untested backup is a hope.

---

10. What maintenance actually looks like

Task	How often	Who	Effort
Update the app	when a release is announced	server owner	Portainer → Stacks → `sc-nav` → **Pull and redeploy**. ~2 min. Because the stack tracks `stable`, this always fetches the newest **release** — never a half-finished commit
Check for a new release	whenever you're in there	org admin	ADMIN → **Server version** panel tells you if one exists. It never self-updates
Game patch moved locations	per patch	org admin	ORG SETTINGS → **Refresh now**. No restart
Commodity prices	automatic	—	Refreshes every 6 h on its own
Nightly backup	automatic	—	The cron job from step 9
Off-site backup copy	monthly	server owner	~5 min
Ubuntu security updates	quarterly	server owner	`sudo apt update && sudo apt upgrade -y && sudo reboot`
Bad imported POI, spam post, wrong data	as needed	org admin	In-app ADMIN panel

That's the whole ongoing commitment: **one click per release, five minutes a month, and a quarterly reboot.**

---

11. Runbook — the things that will actually go wrong

"Login bounces back to the splash / says invalid redirect." Your `OAUTH_REDIRECT_URI` doesn't exactly match the redirect registered on the Discord app. Compare them character by character — `http` vs `https` and a trailing slash both count. Fix whichever is wrong, then redeploy the stack.

"Everyone got logged out after a restart." `SESSION_SECRET` is unset or was changed. Set it in Portainer's env vars and redeploy. The container log says this explicitly on startup.

"The site is down." Portainer → **Containers**. You should see `sc-nav` and `sc-nav-tunnel` both green/running. If `sc-nav` is restarting, click it → **Logs** and read the last 20 lines — misconfiguration says so plainly. If both are running but the site won't load, check the tunnel is **Healthy** in Cloudflare Zero Trust.

"The map only knows a handful of places." The POI catalogs are off. See the 🚨 box in step 8.

"Cloudflare has said 'Pending Nameserver Update' for a day." Almost always one of three things. In order of likelihood: (1) an old nameserver is still listed alongside Cloudflare's at the registrar — it must be *only* Cloudflare's two; (2) the nameservers were typed rather than copied, or copied from another domain — they're assigned per-domain; (3) DNSSEC is still enabled at the registrar. Check what the world actually sees with `dig NS yourorg.com +short`, then compare against the pair on Cloudflare's Overview page. Cloudflare re-checks on its own; there's nothing to click.

"The domain went completely dark — website and email both, `SERVFAIL`." DNSSEC was left on through the nameserver change (5.3). Turn DNSSEC off at Network Solutions immediately; resolution comes back as the old signature expires from caches. This is the reason 5.3 comes before 5.4.

"Org email stopped arriving after the DNS move." An MX or mail-related record didn't make it into Cloudflare, or a `mail` / `smtp` host got left 🟠 proxied. Open your 5.1 inventory and reconcile it against Cloudflare's DNS tab record by record. If you can't fix it in a few minutes, roll back per 5.9 and try again another evening — mail is not something to debug under pressure.

"The subdomain resolves but shows a Cloudflare error page (5xx / 1033)." DNS is fine; the tunnel is the problem. Either the `cloudflared` container isn't running (Portainer → Containers → `sc-nav-tunnel`) or the public hostname points somewhere wrong — it must be service type `HTTP` and URL `sc-nav:8765`, the container name, not an IP.

"We need to undo a bad data import / restore a backup." Stop the stack in Portainer, then on the server:

```
sudo gunzip -c /var/backups/sc-nav/sc_nav-YYYYMMDD-HHMMSS.db.gz > /tmp/restore.db
sudo cp /tmp/restore.db /var/lib/docker/volumes/sc-nav_sc-nav-data/_data/sc_nav.db
```

Start the stack again. (Ask the maintainer first if you're unsure — this overwrites current data.)

"The new release broke something — can we go back?" Partly, and it matters that you read this *before* you need it. Changing the stack's **Reference** to the previous tag (`refs/tags/v1.2.0`) and redeploying puts the old **code** back in about two minutes. What it does **not** undo is the database: releases add columns and tables on startup and never remove them, so a downgraded server can find a database newer than it expects. That is usually harmless and occasionally not.

The safe rollback is therefore **both halves**: repoint to the old tag *and* restore the nightly backup taken before the upgrade, using the recipe just above. Which is the real reason the step-9 cron job matters — the backup from 4 AM this morning is what makes a bad release survivable. Tell the maintainer either way; a release that needs rolling back needs fixing upstream. When you're back on a good version, set the Reference to `refs/heads/stable` again so you resume receiving updates.

---

12. Never do these

- **Never run `docker compose down -v`.** The `-v` deletes the volume — that is your entire database, every survey mark and custom POI the org has ever recorded. Removing the *stack* in Portainer can do the same thing; use **Pull and redeploy** to update, and don't delete the stack.
- **Never point the stack at `refs/heads/main`.** That's the development trunk. It is not broken on purpose, but it is not a release either: it carries half-finished work, and it gets the fixes for its own mistakes an hour later. Your members are on `stable`.
- **Never add `--workers` to the app.** It keeps live position and WebSocket state in memory in one process; a second worker would see half the picture.
- **Never commit or paste the Discord client secret, session secret, or tunnel token** into Discord, a ticket, or the repo. They live only in Portainer's environment variables.
- **Never change nameservers with DNSSEC still enabled** at the registrar — see 5.3. It is the one action in this guide that takes the org's website *and* email fully offline, and the failure is invisible from a browser that has the old answer cached.
- **Never edit the org's DNS at Network Solutions after step 5.** Those screens still accept edits; they just have no effect. All DNS lives in Cloudflare now.
- **Don't drop the UEX refresh below 2 hours.** It's rate-limited upstream and the server enforces the floor anyway.

13. If they outgrow it

The numbers in section 2 say NVMe 4 comfortably covers ~180 members. If the org doubles, the first thing to feel it is memory from breadcrumb trails, and the fix is a one-line constant change (`PATH_MAX` in `server/app.py`) rather than a bigger server. Resizing the VPS one tier is also available and non-destructive. Talk to the maintainer before either.